

Lab 2 110550022 賴柏允

- Implementation Details

Training

```
def train(args):
    # implement the training function here
    if args.model == 'unet':
        model = UNet()
        model_name = "unet"
    else:
        model = ResNet34Unet()
        model_name = "resnet34+unet"

    #print(model)
    model.to('cuda')

    g = torch.Generator()
    g.manual_seed(0)

    dataset = load_dataset(os.path.join(os.getcwd(), '..\\dataset\\oxford-iiit-pet\\'), mode='train')
    dataloader = DataLoader(dataset, batch_size=args.batch_size, shuffle=True, num_workers=1, worker_init_fn=seed_worker, generator=g)

    eval_dataset = load_dataset(os.path.join(os.getcwd(), '..\\dataset\\oxford-iiit-pet\\'), mode='valid')
    eval_dataloader = DataLoader(eval_dataset)

    optimizer = torch.optim.RAdam(model.parameters(), lr=args.learning_rate, decoupled_weight_decay=True, weight_decay=1e-4)
    #scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, factor=0.3, patience=5)
    #scheduler = torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr=5 * args.learning_rate, epochs=args.epochs, steps_per_epoch=len(dataloader))
    criterion = torch.nn.BCEWithLogitsLoss()
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, 5, 0.77)
```

The first section of the training function sets up the training and evaluation data loaders and other training strategies.

The training data loader by default uses `batch_size = 10` due to VRAM limitations on unet (resnet34_unet seems to take up a lot less VRAM).

Shuffle is set to True to introduce more variation to the dataset, and `num_workers` is set to 1 to allow for parallel data loading while keeping the fan noise level low so that I can sleep at night. The last 2 parameters are there to initialize the worker with a seed that's derived from torch's initial seed as seen in

<https://pytorch.org/docs/stable/notes/randomness.html#dataloader> .

The evaluation data loader uses default settings with `num_workers = 0` since the process is fast enough that the additional worker initialization overhead ends up stretching out the time span.

While searching for learning rate schedulers I came across

<https://arxiv.org/pdf/1908.03265> that proposes the RAdam optimizer, which the authors claim is more stable than Adam across a wider range of learning rates. Setting `decoupled_weight_decay` to True makes the optimizer use the

same weight decay method as in AdamW which corresponds more closely to the author's implementation in the RAdam paper. Weight decay is set to prevent the model from overfitting the dataset.

After experimenting with different LR schedulers, the simple stepLR scheduler appears to work best.

```
training_losses = []
valid_losses = []
for epoch in tqdm(range(args.epochs)):
    epoch_loss = 0
    model.train()

    for batch in tqdm(dataloader, desc=f"Epoch {epoch+1}/{args.epochs}"):
        images, labels = batch['image'].to('cuda'), batch['mask'].to('cuda')

        optimizer.zero_grad()

        outputs = model(images)

        assert outputs.shape == labels.shape

        #loss = dice_loss(outputs, labels)
        loss = 0.7 * criterion(outputs, labels) + 0.3 * dice_loss(outputs, labels)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()

    scheduler.step()

    avg_loss = epoch_loss / len(dataloader)
    print(f"Epoch {epoch+1}/{args.epochs} Training loss (BCE+Dice): {avg_loss:.4f}")
    training_losses.append(avg_loss)

    eval_loss = evaluate(model, '', eval_dataloader, 'cuda')
    valid_losses.append(eval_loss.cpu().numpy())
    if eval_loss < lowest_eval_loss:
        lowest_eval_loss = eval_loss
        model_path = f"..\\saved_models\\{model_name}_best.pth"
        torch.save(model.state_dict(), model_path)
        print(f"Best model updated.")
```

In the main training loop, the program iterates through the training data loader “args.epochs” times (default: 50). Model.train() is called inside the epoch loop as suggested in <https://discuss.pytorch.org/t/where-should-i-put-train-and-eval/105009/2>.

In each epoch, the images and masks of each batch are sent to the gpu and the accumulated gradient is cleared out to ensure that the weights are

updated correctly because `loss.backward()` does not clear the gradients itself. The output of the model is obtained and passed into the loss function. which is a combination of binary cross entropy and dice loss. I've tested using only bce, dice and Tversky loss and among the three bce works the best, but I thought that adding the validation metric would guide it to minimize the loss more (and it did). `BCELossWithLogits` is called instead of the plain `BCELoss` function since the model output does not pass through a sigmoid layer.

```
def dice_score(pred_mask, gt_mask):
    # implement the Dice score here
    assert pred_mask.shape == gt_mask.shape
    pred_mask = torch.sigmoid(pred_mask)
    return (2 * torch.sum(pred_mask * gt_mask, dim=(2, 3))) / (torch.sum(pred_mask, dim=(2, 3)) + torch.sum(gt_mask, dim=(2, 3)))

def dice_loss(pred_mask, gt_mask):
    return 1 - dice_score(pred_mask, gt_mask).mean()
```

The dice score is implemented based on the equation listed on Wikipedia.

When applied to Boolean data, using the definition of true positive (TP), false positive (FP), and false negative (FN), it can be written as

$$DSC = \frac{2TP}{2TP + FP + FN}.$$

True positives can be calculated through multiplying the two masks (both the gt and prediction is 1), and the denominator can be written as (TP + FP)

⇒ all positives in the prediction are either correct (TP) or incorrect (FP)

+

(TP + FN)

⇒ all positives in the ground truth mask is either correctly predicted (TP) or missed (FN)

⇒ $2TP + FP + FN$ = sum of positives in the prediction and ground truth.

The loss is calculated by averaging dice scores across the entire batch and deducting the average score from 1.

Even though the dice score is meant for binary masks,

<https://medium.com/data-scientists-diary/implementation-of-dice-loss-vision-pytorch-7eef1e438f68>

3. **Thresholding early:** Avoid hard thresholds (e.g., $\text{pred} > 0.5$) before computing Dice Loss. The loss needs the soft probabilities for backpropagation.

suggests that during training, thresholding shouldn't be applied so that backpropagation works properly.

Afterwards the program performs backpropagation and calls `optimizer.step()` to upgrade the weights and adds the batch loss to the `epoch_loss` variable.

After iterating through the entire dataloader, the scheduler is update by calling `scheduler.step()` and the average loss is calculated. The program then evaluates the model and updates the best one.

Evaluation

```
def evaluate(net, name, data, device):
    # implement the evaluation function here
    net.eval()

    with torch.no_grad():
        eval_loss = 0.0
        for batch in tqdm(data):
            images, lables = batch['image'].to(device), batch['mask'].to(device)
            outputs = net(images)

            eval_loss += dice_loss_quantized(outputs, lables)

        eval_loss /= len(data)

        print(f"{name} Eval Loss: {eval_loss:.4f}")

    return eval_loss
```

The code for evaluation is a simplified version of the training loop.

First `eval()` is called to set the model to evaluation mode, and `torch.no_grad()` is used since the program does not need to keep track of gradients during evaluation (no backpropagation).

The main difference here is that `dice_loss_quantized()` is used instead of `dice_loss (+bce)`

```
def dice_loss_quantized(pred_mask, gt_mask):
    pred_mask = torch.sigmoid(pred_mask) > 0.5
    return 1 - ((2 * torch.sum(pred_mask * gt_mask, dim=(2, 3))) / (torch.sum(pred_mask, dim=(2, 3)) + torch.sum(gt_mask, dim=(2, 3))))).mean()
```

so that thresholding is considered.

Inference

```
if __name__ == '__main__':
    args = get_args()

    device = 'cuda'
    if args.model.startswith('unet'):
        model = UNet()
    else:
        model = ResNet34Unet()

    model.load_state_dict(torch.load(f"..\\saved_models\\{args.model}", weights_only=True))
    model.to(device)

    data_path = args.data_path if os.path.isabs(args.data_path) else os.path.join(os.getcwd(), args.data_path)

    dataset = load_dataset(data_path, mode='test')
    loader = DataLoader(dataset, batch_size=1, shuffle=False)

    model.eval()
    mean, std = torch.tensor(dataset.mean), torch.tensor(dataset.std)
    with torch.no_grad():
        eval_loss = 0.0
        for batch in tqdm(loader):
            image = batch['image'].to(device)
            label = None if args.output else batch['mask'].to(device)

            output = model(image)
            pred_mask = torch.sigmoid(output) > 0.5

            if args.output:
                mask_output = (pred_mask.cpu().numpy().astype(np.uint8) * 255)[0, 0]
                cv2.imwrite(f"..\\outputs\\mask\\mask_{batch[0]['filename']}.png", mask_output)

                reverted_image = image[0].cpu() * std[:, None, None] + mean[:, None, None]
                overlay = (255 * np.flip(overlay_mask(reverted_image, pred_mask[0]), axis=2)).astype(np.uint8)
                cv2.imwrite(f"..\\outputs\\overlay\\overlay_{batch[0]['filename']}.png", overlay)
            else:
                eval_loss += dice_loss_quantized(output, label)

        if not args.output:
            eval_loss /= len(loader)
            print(f"Eval Loss on set (why is there gt data on the testing set): {eval_loss:.4f}")
```

The code for inference is the code for evaluation + image output. For some reason the testing set labels were present so I figured I might as well add an evaluation functionality to the inference code if the `-output` tag is not set.

If the `-output` tag is set, the program outputs the mask as well as an overlaid image. Denormalization is performed before outputting the image so that the colors are reverted.

UNet

```
class UNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.encodeL1 = DoubleConv(3, 64)  # 3 channel image input
        self.encodeL2 = DoubleConv(64, 128)
        self.encodeL3 = DoubleConv(128, 256)
        self.encodeL4 = DoubleConv(256, 512)

        self.decodeL1 = nn.Sequential(
            DoubleConv(512, 1024),
            nn.ConvTranspose2d(1024, 512, kernel_size=2, stride=2)
        )
        self.decodeL2 = nn.Sequential(
            DoubleConv(1024, 512),
            nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
        )
        self.decodeL3 = nn.Sequential(
            DoubleConv(512, 256),
            nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
        )
        self.decodeL4 = nn.Sequential(
            DoubleConv(256, 128),
            nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
        )
        self.decodeL5 = nn.Sequential(
            DoubleConv(128, 64),
            nn.Conv2d(64, 1, kernel_size=1)
        )

    def forward(self, x):
        l1_pre_down = self.encodeL1(x)
        x = F.max_pool2d(l1_pre_down, 2, 2)

        l2_pre_down = self.encodeL2(x)
        x = F.max_pool2d(l2_pre_down, 2, 2)

        l3_pre_down = self.encodeL3(x)
        x = F.max_pool2d(l3_pre_down, 2, 2)

        l4_pre_down = self.encodeL4(x)
        x = F.max_pool2d(l4_pre_down, 2, 2)

        x = self.decodeL1(x)
        x = self.decodeL2(torch.cat((center_crop(l4_pre_down, x.shape[2:]), x), dim=1))
        x = self.decodeL3(torch.cat((center_crop(l3_pre_down, x.shape[2:]), x), dim=1))
        x = self.decodeL4(torch.cat((center_crop(l2_pre_down, x.shape[2:]), x), dim=1))

        return self.decodeL5(torch.cat((center_crop(l1_pre_down, x.shape[2:]), x), dim=1))
```

There weren't a lot of modifications to the unet architecture since the graph is rather clear. The input channels were changed from 1 to 3 to support RGB images.

```

class DoubleConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1, padding_mode='replicate', bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, padding_mode='replicate', bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.block(x)

```

Batchnorm and Relu layers were added since they alleviate gradient explosion and improve training stability.

https://www.reddit.com/r/MachineLearning/comments/fd9whj/d_why_batchnorm_in_unet/
https://www.reddit.com/r/MachineLearning/comments/fd9whj/d_why_batchnorm_in_unet/

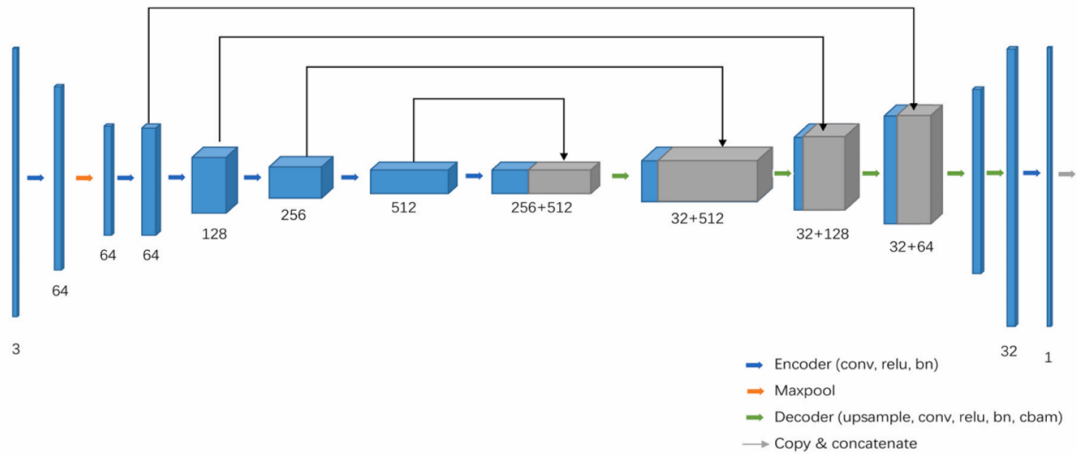
And due to the addition of the batchnorm layer, the biases of the convolution layers can be removed since they no longer contribute to training <https://github.com/kuangliu/pytorch-cifar/issues/52>.

A 1-pixel padding was also added to match the model output size to the ground truth labels since they have the same size as the inputs. Due to the padding, center_crop can actually be removed from the model, but it didn't really affect the training speed, so I didn't bother with it.

ResNet34+UNet

(I won't paste the code here because it's quite similar to code for UNet and it's pretty long)

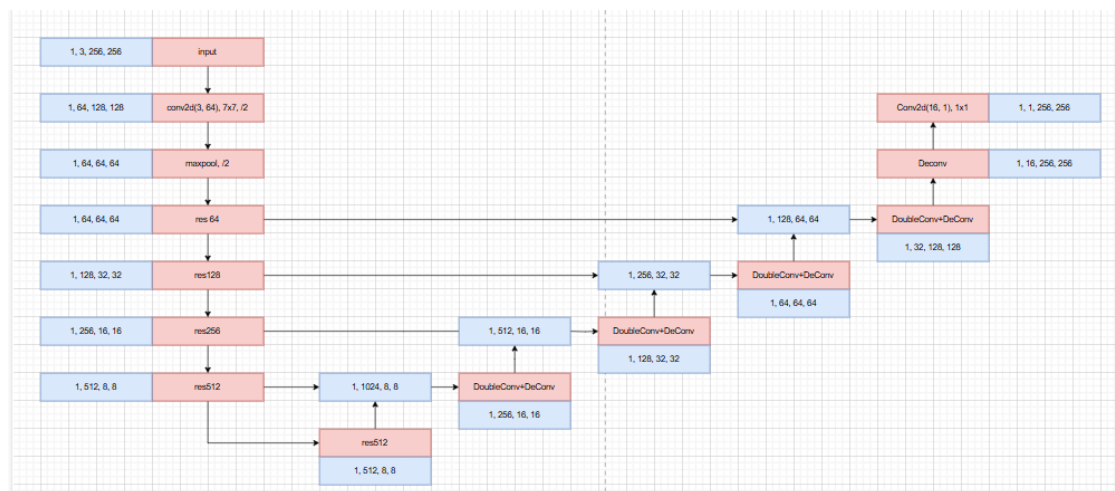
The implementation process of unet with resnet34 backbone wasn't as simple though, since the graph provided by the paper in the lab specs is lacking some vital information and probably incorrect? Either way, the graph is ambiguous, and the paper itself doesn't provide any architectural information either.



The main thing that is lacking here is the mapping of the decoder (input, output channels of the double convolution layer deconvolution layer). In each decoder the input and output size are all different from the normal unet architecture. Channels are divided by 2 in the double conv layer and divided by 2 again in the deconv layer in each decoder in unet, and here it's unclear (768 -> 32, 544 -> 32 ...).

There also seems to be a mistake in the skip connection of the 256-channel encoder. The graph calls for copy and concatenation but uses a 512-channel block to illustrate the output of the copy operation.

Because of this, I had to come up with my own architecture which is shown in the figure below.



I took inspiration from the 512->256 bottleneck layer in the provided graph and instead used a 512->512 bottleneck layer consisting of 3 residual

blocks. The output of the 512 encoder is concatenated with the output of this bottleneck block to form a 1024 channel input, which is of the same channel size as the output of the bottleneck layer in the UNet architecture, so the decoder setup of UNet is directly migrated over. The only difference here is the output layer and this is because of the 4x downscaling of the input layer (conv 7x7, /2 and pool, /2), so I had to add an additional deconvolution layer at the end right before the conv 1x1 to scale the mask back to its original size.

I have no idea if this is optimal tho, but it kind of works.

- Data Preprocessing

I've ditched the SimpleOxfordPetDataset class and implemented the transforms using torchvision since I found working with tensors to be easier.

```
self.mean = [0.4783, 0.4463, 0.3961]
self.std = [0.2631, 0.2575, 0.2655]
self.image_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize((256, 256)),
    transforms.Normalize(mean=self.mean, std=self.std)
])
self.mask_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize((256, 256), interpolation=transforms.InterpolationMode.NEAREST_EXACT)
])
```

```
sample['image'] = self.image_transform(sample['image'])
sample['mask'] = self.mask_transform(sample['mask'])
sample['trimap'] = self.mask_transform(sample['trimap'])
```

The mapping and resize steps were done using ToTensor() and Resize() functions, with the masks and trimaps using nearest_exact interpolation to eliminate any non-binary values. Additionally, the images are normalized using the mean and standard deviation of the testing set which were calculated before implementing the augmentation methods.

```
self.color_jitter = transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.05)
```

```
if self.mode == "train" and random.random() > 0.5:
    image = self.color_jitter(image)
```

```

if self.mode == "train": # only apply agumentation during training
    if random.random() > 0.5:
        sample['image'] = transforms.functional.hflip(sample['image'])
        sample['mask'] = transforms.functional.hflip(sample['mask'])
        sample['trimap'] = transforms.functional.hflip(sample['trimap'])

    if random.random() > 0.5:
        angle = 30 * (random.random() * 2 - 1)
        def random_rotate(img, interp=transforms.InterpolationMode.BILINEAR, resize_interp=transforms.InterpolationMode.BILINEAR):
            upscaled = transforms.functional.resize(img, (336, 336), interpolation=resize_interp)
            return transforms.functional.rotate(upscaled,
                                                angle,
                                                interpolation=interp),
            (256, 256)
        )

        sample['image'] = random_rotate(sample['image'])
        sample['mask'] = random_rotate(sample['mask'], interp=transforms.InterpolationMode.NEAREST, resize_interp=transforms.InterpolationMode.NEAREST_EXACT)
        sample['trimap'] = random_rotate(sample['trimap'], interp=transforms.InterpolationMode.NEAREST, resize_interp=transforms.InterpolationMode.NEAREST_EXACT)

    if random.random() > 0.5:
        top, left = random.randint(0, 32), random.randint(0, 32)
        def random_crop(img, interp=transforms.InterpolationMode.BILINEAR):
            return transforms.functional.resize(
                transforms.functional.crop(img, top, left, 224, 224),
                (256, 256),
                interpolation=interp
            )

        sample['image'] = random_crop(sample['image'])
        sample['mask'] = random_crop(sample['mask'], transforms.InterpolationMode.NEAREST_EXACT)
        sample['trimap'] = random_crop(sample['trimap'], transforms.InterpolationMode.NEAREST_EXACT)

    if random.random() > 0.7:
        sample['image'] = transforms.functional.gaussian_blur(sample['image'], kernel_size=3)

    if random.random() > 0.7:
        noise = torch.randn_like(sample['image']) * 0.07
        sample['image'] = torch.clamp(sample['image'] + noise, 0, 1)

```

I've implemented several data augmentation methods which are applied randomly on-the-fly during training. The techniques include:

1. Horizontal flips

Vertical flips are unrealistic for most images

2. Rotation

An angle is picked randomly in the range of [-30, 30] degrees.

The image is enlarged before rotation and cropped back down to remove the black edges.

The masks and trimaps are interpolated using nearest for the same reason as the input resize.

3. Zoom+crop

A 224x224 sub-image is cropped randomly from the image and resized back up.

4. Gaussian blur

5. Gaussian noise

Another thing I found while training the model was that there were blank masks in the training set which negatively affects the training process.

```

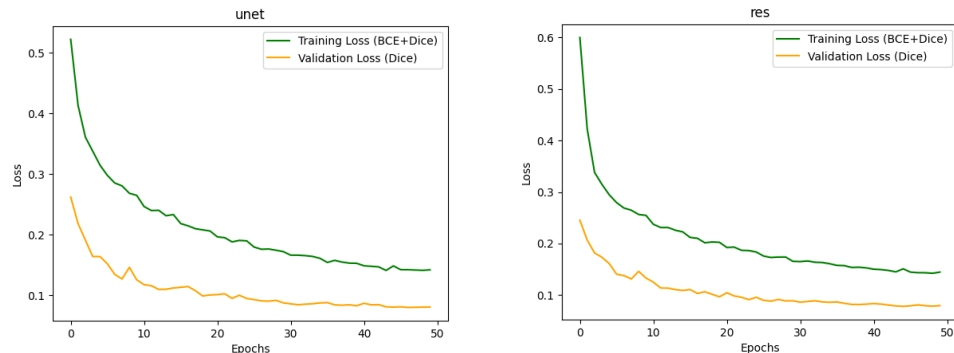
blacklist = ['saint_bernard_108', 'Egyptian_Mau_162', 'Egyptian_Mau_196', 'Egyptian_Mau_165', 'leonberger_18',
             'miniature_pinscher_14', 'saint_bernard_15']

filenames = [x.split(" ")[0] for x in split_data if x.split(" ")[0] not in blacklist]

```

These files were removed from the filenames list.

- Analyze the experiment results



For the first few epochs, the loss for the resnet+unet model decreases more rapidly. The final accuracy improvement isn't noticeable, however.

One thing worth noting is that the performance of resnet+unet is a lot better compared to unet, with more than double the iterations per second during training for some reason. It also takes less resources so batch size can be pushed even further.

Other tests:

1. Optimizers:

I've only tested Adam and RAdam, and RAdam does seem more stable during training, and adding weight decay helped alleviate overfitting, especially on the resnet34+unet model since it has a lot more parameters.

2. Schedulers:

I've tested:

Cosine annealing with warmup (<https://github.com/katsura-jp/pytorch-cosine-annealing-with-warmup>)

Suggested by the top comment in

https://www.reddit.com/r/MachineLearning/comments/oy3co1/d_how_to_pick_a_learning_rate_scheduler/

The final accuracy was bad, likely because of the amount of hyperparameters needed.

One cycle LR

Since both the RAdam paper and the reddit post above recommended having a warmup phase, and the rear half of the scheduler follows the general learning rate decay.

It didn't really make a difference compared to StepLR.

Reduce LR on plateau

I used the validation loss as the metric for this, but the validation loss ends up oscillating (probably due to high lr) and rendered the scheduler useless.

StepLR

The least fancy method that surprisingly worked the best.

3. Losses:

I've tested:

Dice Loss, which is $1 - \text{dice_score.mean}()$

Tversky Loss with $\alpha = 0.4$ and $\beta = 0.6$ suggested in

<https://arxiv.org/pdf/2312.05391> (The paper didn't provide the beta value though)

BCE Loss

Out of the three losses, BCE loss performed the best in dice score evaluation but struggled to lower the training loss in the later epochs. So the final loss function I chose was a composite loss combining $0.4 * \text{dice}$ and $0.6 * \text{bce}$ loss. This combination allowed me to reach slightly over 92% accuracy on the testing set with both models.

Others:

Batch size was set to the maximum my hardware can handle which is 10.

However, I found that UNet is taking a lot more resources than Resnet+Unet so I could push it further for resnet but decided to keep them the same.

- **Execution steps**

The environment is conda with python 3.12.9. It also worked fine for python 3.8 but the versions in requirements.txt might need to be removed.

First cd into the src directory, then:

For training, run

```
python train.py -m {unet, res}
```

to train unet or resnet+unet with default parameters (batch size = 10 for 50 epochs)

Evaluation will be run after each epoch during training, but to run evaluation independently, run

```
python evaluation.py
```

This evaluates every model in the saved_models folder and outputs the best one (the program used to save a checkpoint every 5 epochs, but now the only saved models will be the best for both unet and res+unet)

For inference, run

```
python inference.py --model {unet_best.pth, resnet34+unet_best.pth,  
<model_name>} [--output]
```

This tests the model on the testing split of the oxford pet dataset. --data_path can be specified but I'm not sure if the implementation will work correctly with a different dataset since the specifics weren't mentioned.

If `--output` is set, the program will output the masks and overlayed images in the `outputs/mask` and `outputs/overlay` folders. If `--output` is not set, the program will assume that ground truth labels are provided and will run evaluation rather than inference.

- **Discussion**

The main improvement that can be made is hyperparameter tuning since the timespan for this assignment is rather limited.

The dataset can be expanded by adding images from other datasets, and the dataset itself could use some verification because I've found a few blank masks just by looping over the dataset.

Another topic may be the optimization of how resnet and unet are merged as there doesn't seem to be a consensus on how the network should look like out of the 5 or so papers I've seen. There also isn't much reasoning that goes into the network design, as if the architecture was created from a gut feeling rather than a rigorous procedure. I've seen some comments on online forums saying that ML has become more of an empirical science subject because a lot of researchers will just mess around with the configurations until they find one that works well and come up with an explanation afterwards.